



F107 – Foundations of Computer Science

Lecture 2

Dr. Sanjit Kumar Saha

Topics



- Operations on Data
 - Logical
 - Shift
 - Arithmetic
- Data
 - Types, Storage, and Processing
 - Text
 - Numbers
 - Images
 - Audio
 - Video
 - Raw data
- Book: Chapters 3 and 4

Logical Operations



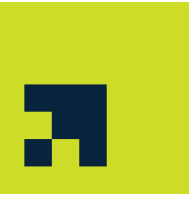
- Logic operations apply on
 - individual bits of a pattern, or
 - two corresponding bits in two patterns.
- A logic operation at the pattern level is n logic operations, of the same type, at the bit level
 - n is the number of bits in the pattern.

Logical Operations at Bit Level

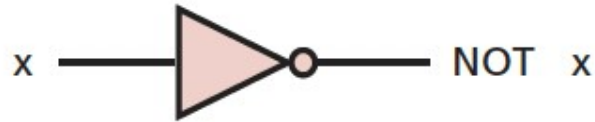


- A bit can take one of the two values: 0 or 1.
 - 0 = false
 - 1 = true
- To manipulate bits, four bit-level operations are
 - NOT
 - AND
 - OR
 - XOR

NOT

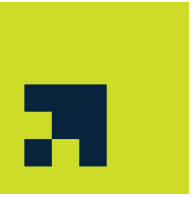


- The NOT operator is a unary operator:
 - it takes only one input.
 - the output bit is the complement of the input

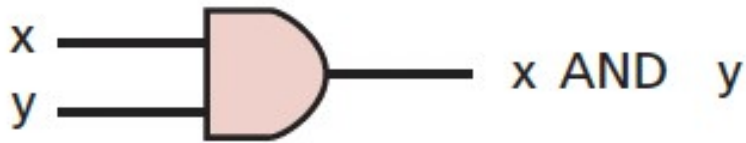


<i>x</i>	<i>NOT x</i>
0	1
1	0

AND

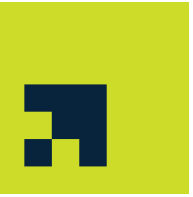


- The AND operator is a binary operator:
 - it takes two inputs.
 - the output bit is 1 if both inputs are 1s and
 - the output is 0 in the other three cases

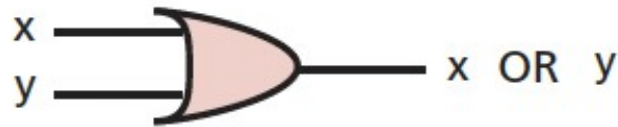


<i>x</i>	<i>y</i>	<i>x AND y</i>
0	0	0
0	1	0
1	0	0
1	1	1

OR



- The OR operator is a binary operator:
 - it takes two inputs.
 - the output bit is 0 if both inputs are 0s and
 - the output is 1 in the other three cases

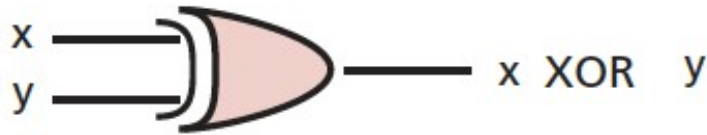


<i>x</i>	<i>y</i>	<i>x OR y</i>
0	0	0
0	1	1
1	0	1
1	1	1

XOR



- The XOR operator is a binary operator:
 - it takes two inputs.
 - the output is 0 when both inputs are the same
 - the output is 1 when the inputs are different



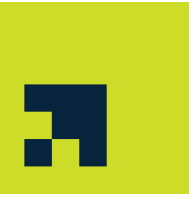
<i>x</i>	<i>y</i>	<i>x XOR y</i>
0	0	0
0	1	1
1	0	1
1	1	0

Example

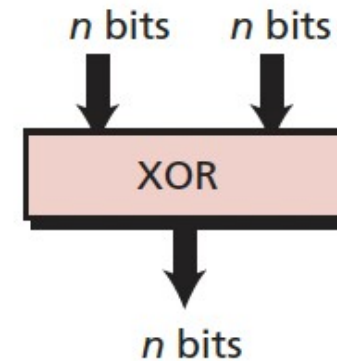
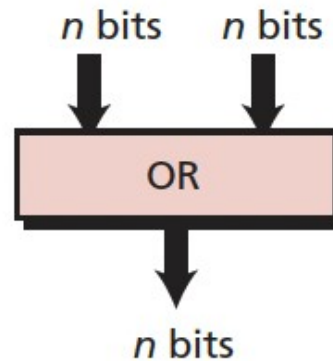
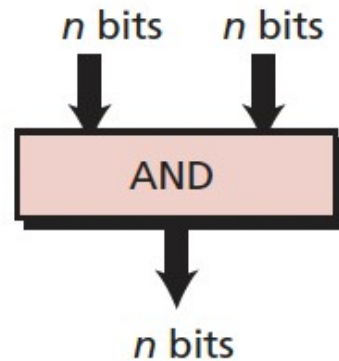
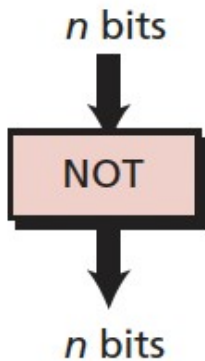


- In English we use the conjunction “or” sometimes to mean an inclusive-or, and sometimes to mean an exclusive-or.
 - a. The sentence “I would like to have *a car* **or** *a house*” uses “*or*” in the *inclusive* sense—I would like to have a car, a house or both.
 - b. The sentence “Today is either *Monday* **or** *Tuesday*” uses “*or*” in the *exclusive* sense—today is either Monday or Tuesday, but it cannot be both.

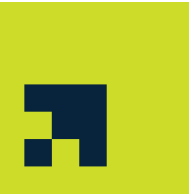
Logic Operations at Pattern Level



- The same four operators (NOT, AND, OR, and XOR) can be applied to an n -bit pattern.
- The effect is the same as applying each operator to each individual bit for NOT and to each corresponding pair of bits for the other three operators.



Logic Operations at Pattern Level



- Use the NOT operator on the bit pattern 10011000.

NOT	1	0	0	1	1	0	0	0	<i>Input</i>
									<i>Output</i>

- Use the AND operator on the bit patterns 10011000 and 00101010.

AND	1	0	0	1	1	0	0	0	<i>Input 1</i>
	0	0	1	0	1	0	1	0	<i>Input 2</i>
									<i>Output</i>

Logic Operations at Pattern Level



- Use the OR operator on the bit patterns 10011001 and 00101110.

OR	1	0	0	1	1	0	0	1	<i>Input 1</i>
	0	0	1	0	1	1	1	0	<i>Input 2</i>
									<i>Output</i>

- Use the XOR operator on the bit patterns 10011001 and 00101110.

AND	1	0	0	1	1	0	0	1	<i>Input 1</i>
	0	0	1	0	1	1	1	0	<i>Input 2</i>
									<i>Output</i>

Applications



- The four logic operations can be used to modify a bit pattern.
 - Complementing (NOT)
 - Unsetting (AND)
 - Setting (OR)
 - Flipping (XOR)

Complementing (NOT)



- The only application of the NOT operator is to complement the whole pattern.
- Applying this operator to a pattern changes every 0 to 1 and every 1 to 0.
- This is sometimes referred to as a *one's complement* operation.

Unsetting (AND)

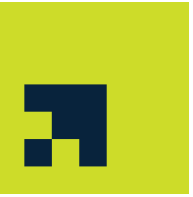


- The AND operator unset (force to 0) specific bits in a bit pattern.
- The second input in this case is called a mask.
- Use a mask to *unset* (clear) the five leftmost bits of a pattern. Test the mask with the pattern 10100110.

Solution: The mask is 00000111. The result of applying the mask is:

AND	1	0	1	0	0	1	1	0	<i>Input 1</i>
	0	0	0	0	0	1	1	1	<i>Mask</i>
	0	0	0	0	0	1	1	0	<i>Output</i>

Setting (OR)

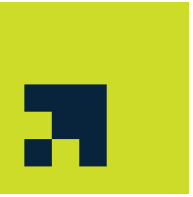


- The OR operator set (force to 1) specific bits in a bit pattern.
- Again we can use a mask, but a different one.
- Use a mask to *set* the five leftmost bits of a pattern. Test the mask with the pattern 10100110.

Solution: The mask is 11111000. The result of applying the mask is:

OR	1	0	1	0	0	1	1	0	<i>Input 1</i>
	1	1	1	1	1	0	0	0	<i>Mask</i>
	1	1	1	1	1	1	1	0	<i>Output</i>

Flipping (XOR)

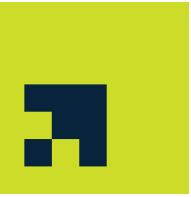


- The XOR operator is to flip (complement) specific bits in a bit pattern.
- Again we can use a mask, but a different one.
- Use a mask to *flip* the five leftmost bits of a pattern. Test the mask with the pattern 10100110.

Solution: The mask is 11111000. The result of applying the mask is:

OR	1	0	1	0	0	1	1	0	<i>Input 1</i>
	1	1	1	1	1	0	0	0	<i>Mask</i>
	0	1	0	1	1	1	1	0	<i>Output</i>

Exercise



- Which logical operation do you have to use to mathematically express:
 - You only receive a Bachelor degree if you successfully pass all courses
 - Either use butter or margarine to bake cake
 - You freeze rock solid if you in one of these places: in space, in a refrigerator, arctic
 - The opposite of daytime is nighttime
- What are the results of the following bit-level operations?
 - $\neg(1 \wedge 0 \vee 1) \wedge (0 \vee 0 \vee 0 \vee 1 \wedge 0)$
 - $10100111 \wedge 01101101$
 - $(101101 \oplus 100001) \wedge 111000 \vee 000100$
- What is $\neg 20_{10}$? Does it make sense?

Shift Operations



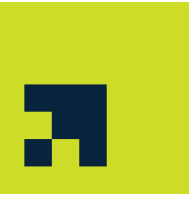
- Shift operations *move the bits* in a pattern, *changing the positions* of the bits.
- They can move bits to the *left* or to the *right*.
- Two categories:
 - logical shift operations
 - arithmetic shift operations

Logical Shift Operations

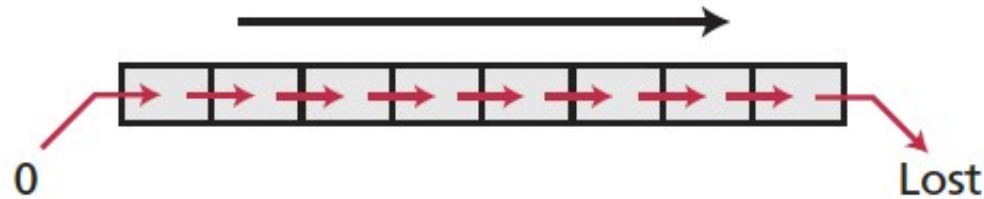


- A logical shift operation is applied to a pattern that does not represent a signed number.
- The reason is that these shift operations may change the sign of the number that is defined by the leftmost bit in the pattern.
- Two types of logical shift operations:
 - Logical shift
 - Logical circular shift (Rotate)

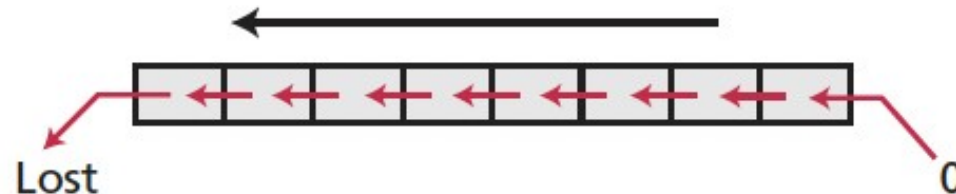
Logical Shift Operations



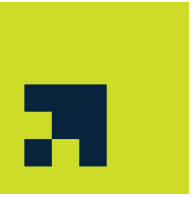
- A simple *right shift* operation shifts each bit one position to the right.
- In an n -bit pattern, the rightmost bit is lost and a 0 fills the leftmost bit.



- A simple *left shift* operation shifts each bit one position to the left.
- In an n -bit pattern, the leftmost bit is lost and a 0 fills the rightmost bit.

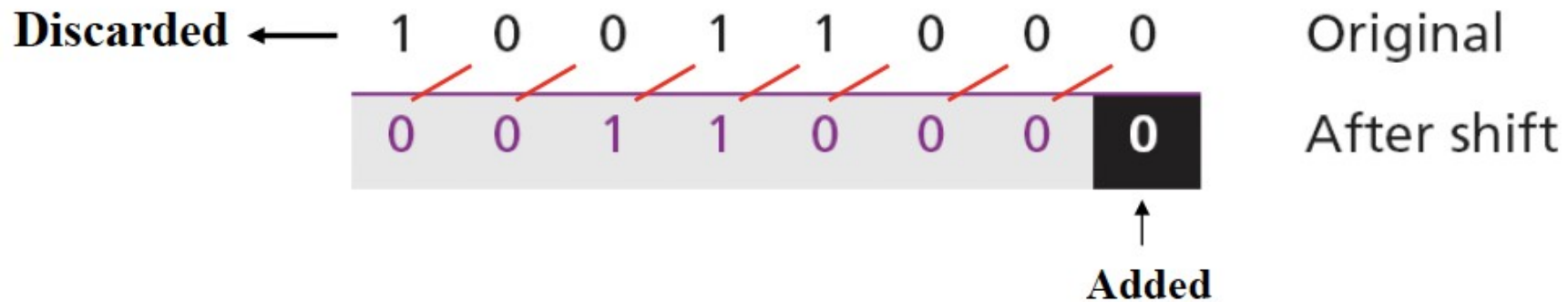


Logical Left Shift Operations

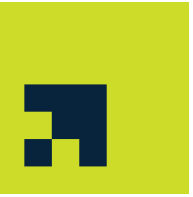


- Use a logical left shift operation on the bit pattern 10011000.

Solution: The solution is shown below. The leftmost bit (white in the black background) is lost and a 0 is inserted as the rightmost bit (the bit in color):



Logical Right Shift Operations

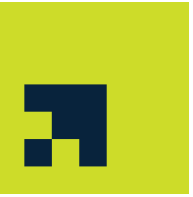


- Use a logical right shift operation on the bit pattern 10011000.

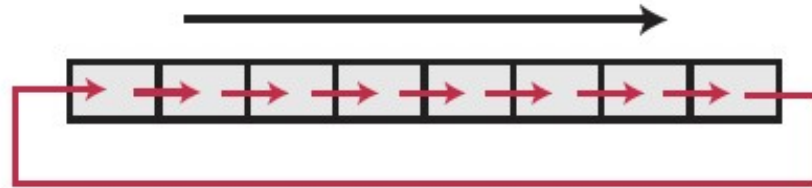
Solution:

1	0	0	1	1	0	0	0	<i>Original</i>	→
								<i>After Shift</i>	

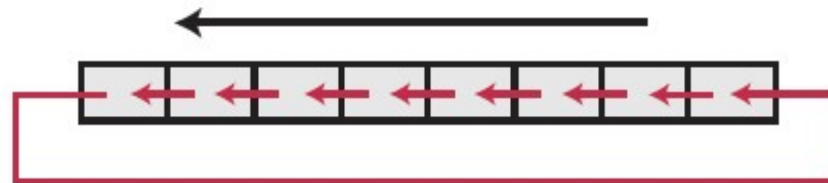
Circular Shift Operations



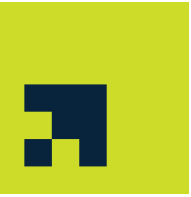
- A circular shift operation (or rotate operation) shifts bits, but no bit is lost or added.
- A ***circular right shift*** (or right rotate) shifts each bit one position to the right. The rightmost bit is circulated and becomes the leftmost bit.



- A ***circular left shift*** (or left rotate) shifts each bit one position to the left. The leftmost bit circulates and becomes the rightmost bit.

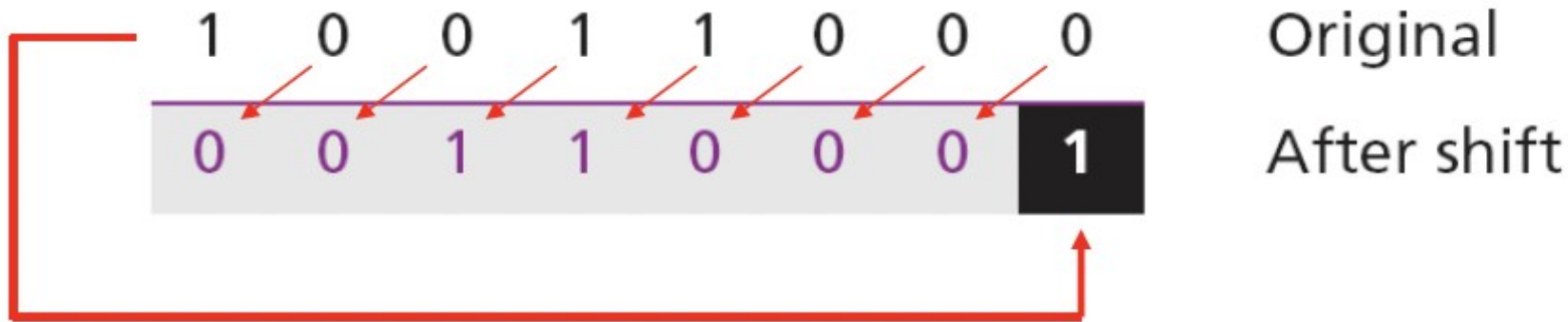


Circular Left Shift Operations



- Use a circular left shift operation on the bit pattern 10011000.

Solution: The solution is shown below. The leftmost bit (the white bit in the black background) is circulated and becomes the rightmost bit:



Circular Right Shift Operations

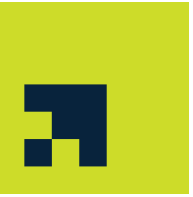


- Use a circular right shift operation on the bit pattern 10011000.

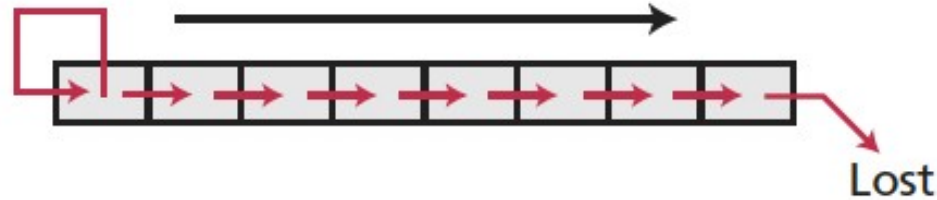
Solution:

<i>Circular</i>	1	0	0	1	1	0	0	0	<i>Original</i>
									<i>After Shift</i>

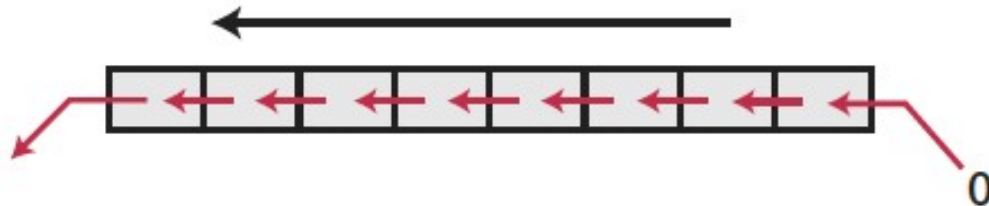
Arithmetic Shift Operations



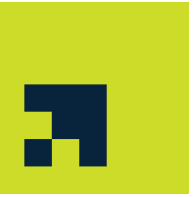
- Arithmetic shift operations assume that the bit pattern is a signed integer in *two's complement* format.
 - arithmetic right shift is used to divide an integer by two



- arithmetic left shift is used to multiply an integer by two

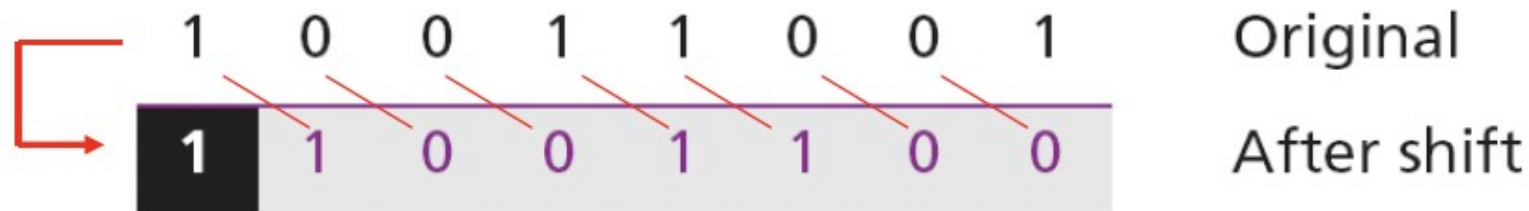


Arithmetic Right Shift Operations



- Use an arithmetic right shift operation on the bit pattern 10011001. The pattern is an integer in two's complement format.

Solution: The solution is shown below. The leftmost bit is retained and also copied to its right neighbor bit.



- The original number was **-103** and the new number is **-52**, which is the result of dividing **-103** by **2** truncated to the smaller integer.

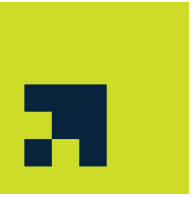
Arithmetic Left Shift Operations



- Use an arithmetic left shift operation on the bit pattern 01111111. The pattern is an integer in two's complement format.

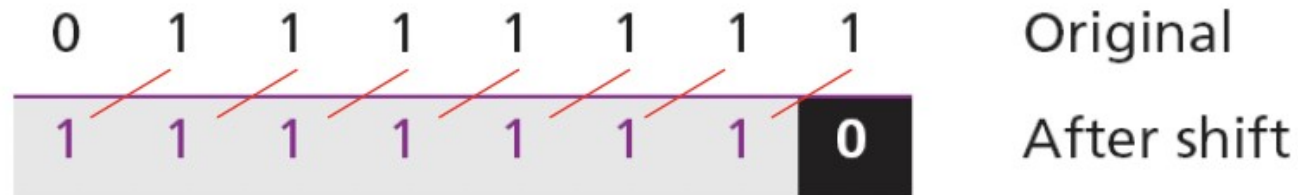
Solution:

Arithmetic Left Shift Operations



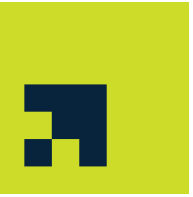
- Use an arithmetic left shift operation on the bit pattern 01111111. The pattern is an integer in two's complement format.

Solution: The solution is shown below. The leftmost bit is lost and a 0 is inserted as the rightmost bit.



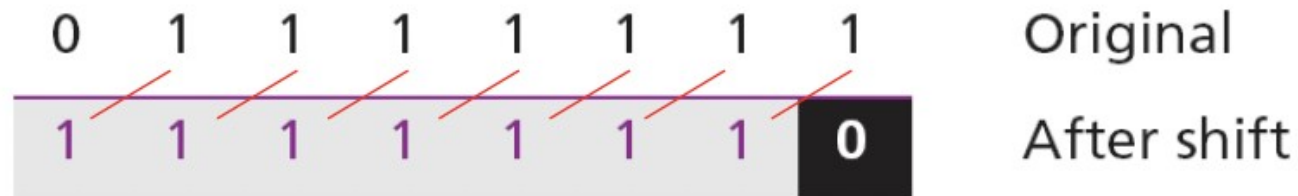
- The original number was **127** and the new number is **-2**.

Arithmetic Left Shift Operations



- Use an arithmetic left shift operation on the bit pattern 01111111. The pattern is an integer in two's complement format.

Solution: The solution is shown below. The leftmost bit is lost and a 0 is inserted as the rightmost bit.



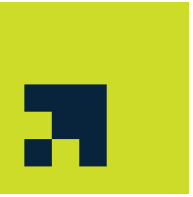
- The original number was **127** and the new number is **-2**.
- Here the result is **not valid** because an **overflow** has occurred. The expected answer **$127 \times 2 = 254$** cannot be represented by an **8-bit** pattern.

Exercise



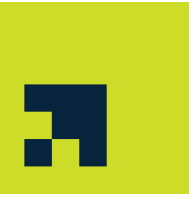
- Shift the pattern 1000 1111 0001 by
 - Three positions to the right using a simple shift
 - Four positions to the left using a rotate
 - Four positions to the right using an arithmetic shift

Arithmetic Operations



- Arithmetic operations involve
 - adding, subtracting, multiplying, and dividing
- We can apply these operations to
 - integers and floating-point numbers
- Arithmetic operations on integers
 - All arithmetic operations can be applied to integers.
 - Although multiplication (division) can be implemented using repeated addition (subtraction), the procedure is not efficient.
 - There are more efficient procedures for multiplication and division, such as Booth procedures.
 - We only discuss addition and subtraction of integers.

Two's Complement Integers

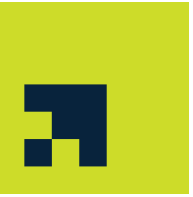


- When the subtraction operation is encountered, the computer simply changes it to an addition operation, but makes two's complement of the second number.
- In other words:

$$A - B \leftrightarrow A + (\overline{B} + 1)$$

Where $\overline{B}$ is the one's complement of B and
 $(\overline{B} + 1)$ means the **two's complement of B**

Addition



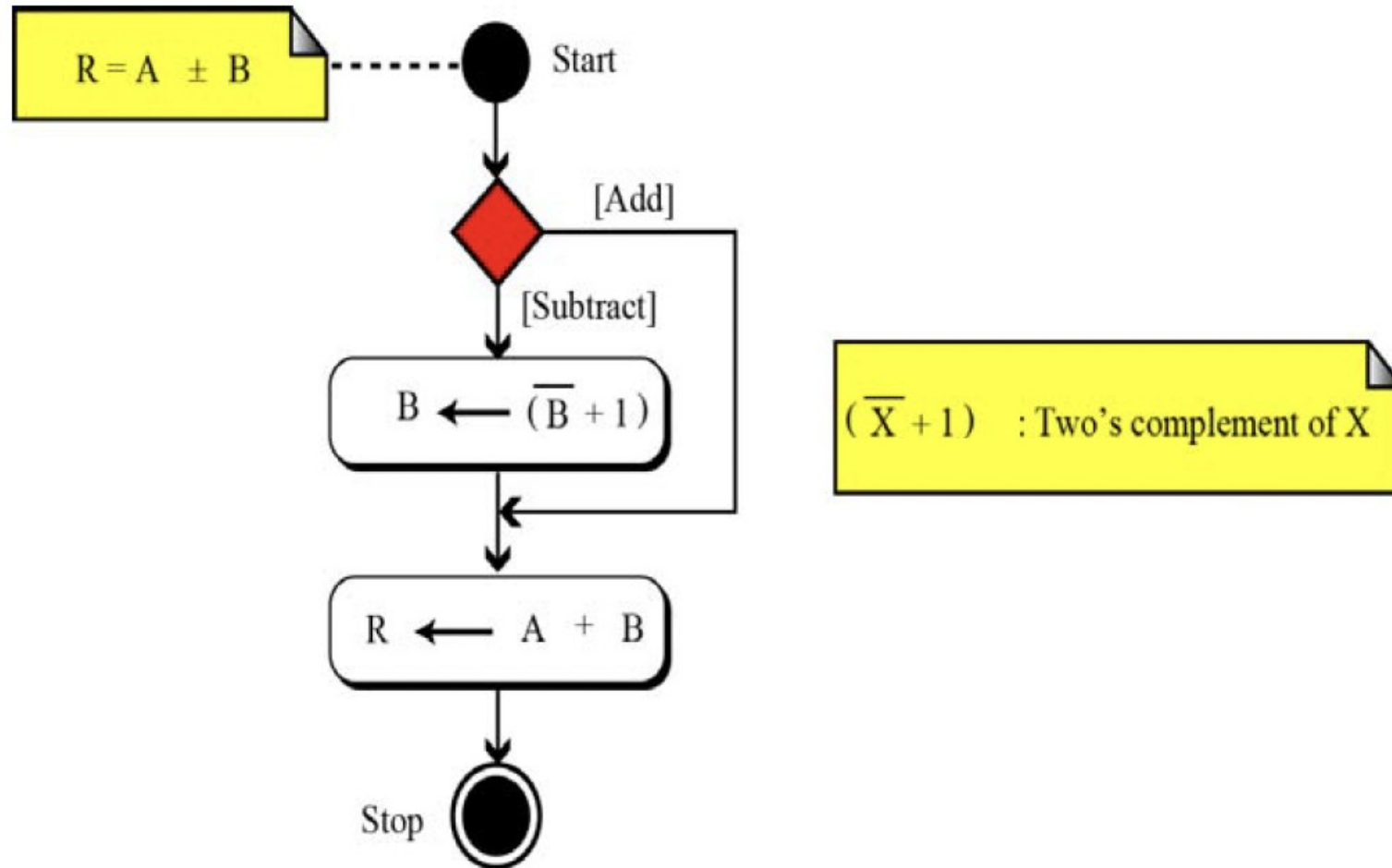
- We should remember that we add integers column by column. The following table shows the sum (S) and carry (C).

Carry and sum resulting from adding two bits

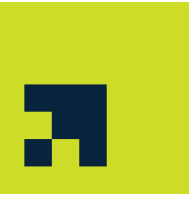
<i>Column</i>	<i>Carry</i>	<i>Sum</i>	<i>Column</i>	<i>Carry</i>	<i>Sum</i>
Zero 1s	0	0	Two 1s	1	0
One 1	0	1	Three 1s	1	1

Addition...

- Addition and subtraction of integers in two's complement format



Example



- Two integers A and B are stored in two's complement format. Show how B is added to A.

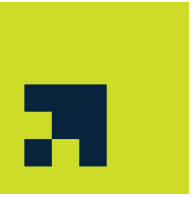
$$A = (00010001)_2 \quad B = (00010110)_2$$

Solution:

The operation is adding. A is added to B and the result is stored in R.

									Carry
	0	0	0	1	0	0	0	1	A
+	0	0	0	1	0	1	1	0	B
									R

Example



- Two integers A and B are stored in two's complement format. Show how B is added to A.

$$A = (00010001)_2 \quad B = (00010110)_2$$

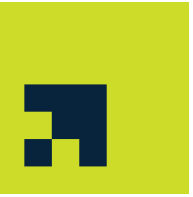
Solution:

The operation is adding. A is added to B and the result is stored in R.

			1						Carry
	0	0	0	1	0	0	0	1	A
+	0	0	0	1	0	1	1	0	B
	0	0	1	0	0	1	1	1	R

We check the result in decimal: $(+17) + (+22) = (+39)$

Example



- Two integers A and B are stored in two's complement format. Show how B is added to A:

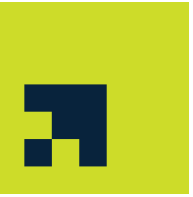
$$A = (00011000)_2 \quad B = (11101111)_2$$

Solution:

The operation is adding. A is added to B and the result is stored in R. Note that the last carry is discarded because the size of the memory is only 8 bits:

			1						Carry
	0	0	0	1	1	0	0	0	A
+	1	1	1	0	1	1	1	1	B
									R

Example



- Two integers A and B are stored in two's complement format. Show how B is added to A:

$$A = (00011000)_2 \quad B = (11101111)_2$$

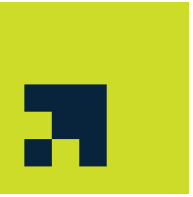
Solution:

The operation is adding. A is added to B and the result is stored in R. Note that the last carry is discarded because the size of the memory is only 8 bits:

1	1	1	1	1					Carry
	0	0	0	1	1	0	0	0	A
+	1	1	1	0	1	1	1	1	B
	0	0	0	0	0	1	1	1	R

Checking the result in decimal, $(+24) + (-17) = (+7)$

Exercise



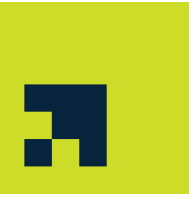
- Two integers A and B are stored in two's complement format. Show how B is subtracted from A:

$$A = (00011000)_2 \quad B = (11101111)_2$$

Solution:

									Carry
	0	0	0	1	1	0	0	0	A
+									B
									R

Exercise...



- Two integers A and B are stored in two's complement format. Show how B is subtracted from A:

$$A = (00011000)_2 \quad B = (11101111)_2$$

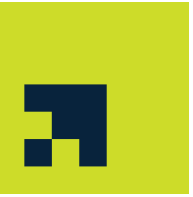
Solution:

The operation is subtracting. A is added to $(B^c + 1)$ and the result is stored in R:

			1						Carry
	0	0	0	1	1	0	0	0	A
+	0	0	0	1	0	0	0	1	B^c+1
	0	0	1	0	1	0	0	1	R

Checking the result in decimal, $(+24) - (-17) = (+41)$.

Exercise



- Two integers A and B are stored in two's complement format. Show how B is subtracted from A:

$$A = (11011101)_2 \quad B = (00010100)_2$$

Solution:

									Carry
	0	0	0	1	1	0	0	0	
+									
									R

Data and Data Types



- All data stored as bits, lots and lots of them
- Classification according to content
 - Text
 - Numbers
 - Images
 - Audio
 - Video
 - “Raw” data
- Data types depend on programming language
- Content representation in files
 - Text files: can be viewed and edited with text editor
 - Binary files: special format, preceded with special “header”

Text Data



- Text must be encoded

- ASCII: 7 bit encoding

- 128 symbols:

- 26 Latin characters (lower and upper case)
 - 10 Arabic numerals
 - Some special printable characters
 - Many special non-printable characters (EOL, EOF, NUL, SP, NBSP, ...)

Code	...0	...1	...2	...3	...4	...5	...6	...7	...8	...9	...A	...B	...C	...D	...E	...F
0...	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1...	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2...	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3...	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4...	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5...	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6...	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7...	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

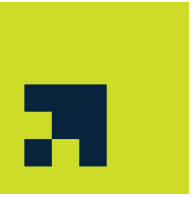
- ISO8859: 8 bit encoding

- Various regional variants, covering different letters

- Unicode: 32 bit encoding

- UTF-8 is de facto standard in Internet
 - Contains all present and almost all previous languages, even smileys and simple pictograms
 - These are “letters”, not “pictures”: kW , 遮 , ✕, ㄣ, ཨྱ, ☠, ☙, 🕒, ☒, ☢, ڇ

Text Data



- Typical data types in programming languages
 - character, string, text
- Storage
 - Binary format (raw data), e.g. doc, pdf, pptx
 - Text format
 - Unstructured, e.g. txt, md
 - Semi-structured, e.g. csv, ini, json
 - Structured, e.g. xml, html, yaml, tex
- File sizes moderate
 - Compression possible
 - Detect and replacement patterns

```
// TXT
John, 27, is alive, tel(home):+2125551234, tel(office):
+6465554567, unmarried
```

```
// JSON
{
  "first_name": "John",
  "is_alive": true,
  "age": 27,
  "phone_numbers": [
    { "type": "home", "number": "212 555-1234" },
    { "type": "office", "number": "646 555-4567" } ],
  "spouse": null
}
```

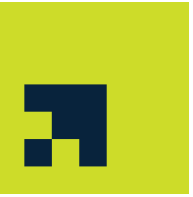
```
// XML
<?xml version="1.0" encoding="UTF-8" ?>
<root xmlns="https://path-to-schema-file.xsd">
  <first_name>John</first_name>
  <is_alive>true</is_alive>
  <age>27</age>
  <phone_numbers>
    <type>home</type>
    <number>212 555-1234</number>
  </phone_numbers>
  <phone_numbers>
    <type>office</type>
    <number>646 555-4567</number>
  </phone_numbers>
  <spouse></spouse>
</root>
```

Exercise



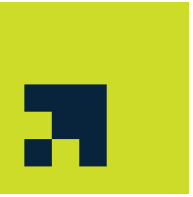
- Represent the word “Gisma” in an ASCII text file.
 - Mind the special characters.

Number Data



- Unsigned and signed integers
 - Unsigned integers: only non-negative values
 - Signed integers:
 - Sign-and-Magnitude
 - MSB represents sign (0 is positive, 1 is negative)
 - Extra: Two's complement
- Data type defines supported range of numbers
 - Typically 1bit, 8bit, 16bit, 32bit, or 64bit
 - `bool`, `char`, `short int`, `int`, `long`, `long long`
 - `unsigned char`, `unsigned short int`, `unsigned int`, `unsigned long`
 - Sign may occupy a bit
 - Overflow/underflow may occur
 - If used data type cannot carry result of operation
 - Avoidance: specifically cast the type into the next larger type (if necessary)

Exercise



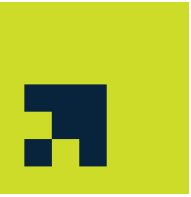
- Represent the two numbers as 8bit unsigned numbers and add them up:
 - $50 + 66 =$
 - $150 + 100 =$
- Represent the number 5000, -5000, and -6666 as 16bit S-a-M integers
- Which number is behind the following 8bit S-a-M integer: 10101010 ?

Exercise



- How is the number 8944 stored in a 16bit integer?
- How is the number -8944 stored in a 16bit integer?
- Which number is in this 8bit integer stored: 0101 1101 ?
- Which number is in this 8bit integer stored: 1001 1101 ?

Extra: Number Data – Two's Complement



- Default way to represent, store, and process signed integers
 - Whole number range is divided into two (almost) equal subranges, e.g. for 4bit:
 - 0000 to 0111 represents 0 through 7
 - 1111 to 1000 represents -1 through -8
 - Supported number range of an n -bit number: $\{-2^{n-1}, \dots, 2^{n-1} - 1\}$
- One's Complement of a number
 - Negate all bits of number individually
 - Original value obtained if applied twice
- Two's Complement of a number
 - Take One's Complement of the number, then add 1 in binary
 - Overflow detection in addition: if two left-most carry bits are not the same
- Why? Simplified addition and subtraction 😊
 - Subtraction can be changed into addition: $A - B = A + B^*$
 - Addition: As we know it with carries, except for the highest position's carry: just drop it

Extra: Exercise



- Calculate $8 + 20$ in 8bit Two's Complement notation!
- Calculate $6 - 3$ in 8bit Two's Complement notation!
- Calculate $-5 - 20$ in 8bit Two's Complement notation!
- Subtract 70 from 50 in 8bit Two's Complement notation!

Image Data

- How is a picture constructed?
 - Rasterized: grid of pixels
 - Pixel is smallest image unit in terms of size
 - Color model and color depth describe color of pixels
 - Zoom limited by pixel grid
 - Vector: description of geometric objects
 - Shape, size, position, angle, fill, and outline properties of geometric objects
 - Infinite zoom possible
 - Meta data in beginning of file
- Picture size considerable!
 - Compression: PNG, JPEG, GIF, WEBP, ...
 - Replace picture parts and recurring patterns
 - Psychovisual coding, Remove unused colors (palette), ...
 - Lossy vs. lossless

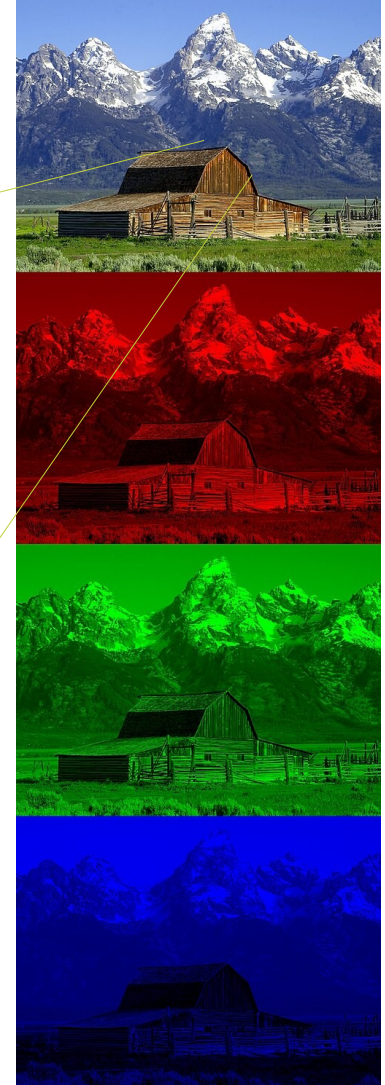
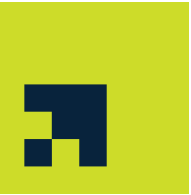


Image Data...



- Color representation
 - Each pixel or shape has a certain mixed color
 - Color components are represented in bits

Bit per Pixel	Number of colors	Use cases
1	black and white	first monochrome screens
4	16	Atari, Commodore64
6	50-64	NES (54 colors)
16	65536	IBM XGA connector
24	~16M	Typical DVI/HDMI, majority of screens
48/64	~281T, alpha optional	Internal handling in graphics SW



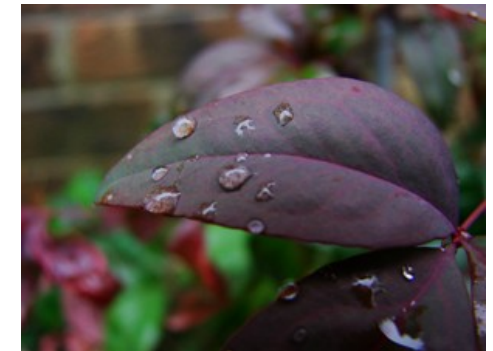
1bit (4kB)



4bit (13kB)



8bit (37kB)



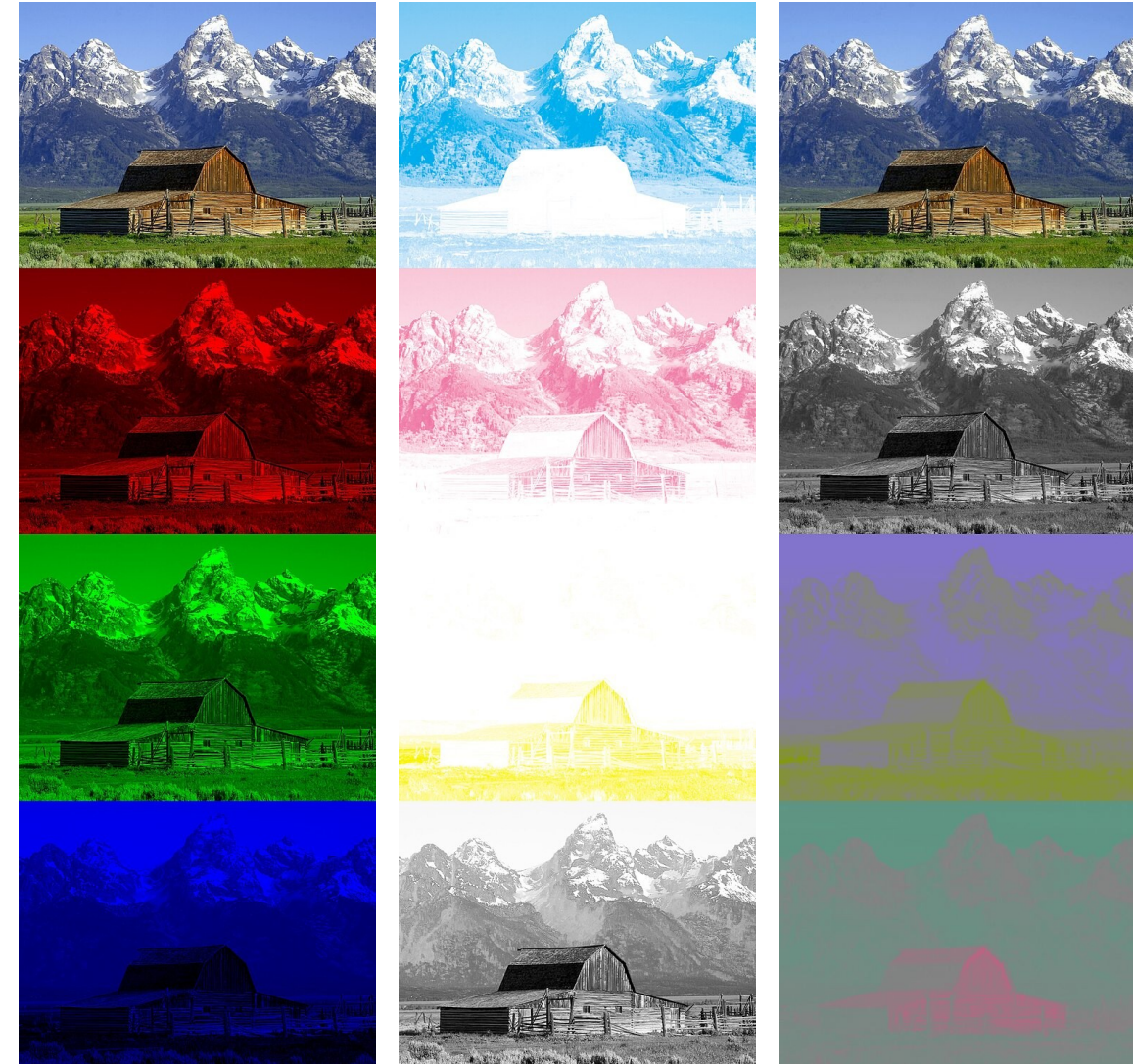
24bit (98kB)

Image Data...

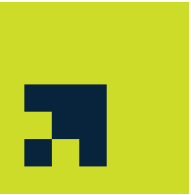
- Color components
 - RGB w/ or w/o Alpha channel
 - Additive approach
 - Alpha channel: Transparency
 - CMY(K)
 - Subtractive approach
 - YCbCr
 - Luminance-Chrominance approach

Some colors defined in True-Color

Color	Red	Green	Blue	Color	Red	Green	Blue
Black	0	0	0	Yellow	255	255	0
Red	255	0	0	Cyan	0	255	255
Green	0	255	0	Magenta	255	0	255
Blue	0	0	255	White	255	255	255



Color Image



- This image comprises many different colors. Almost all colors can be generated from the three primary colors – Red, Green, and Blue. Therefore, we can say that each colored image is a unique composition of these three colors or 3 channels – Red, Green, and Blue.



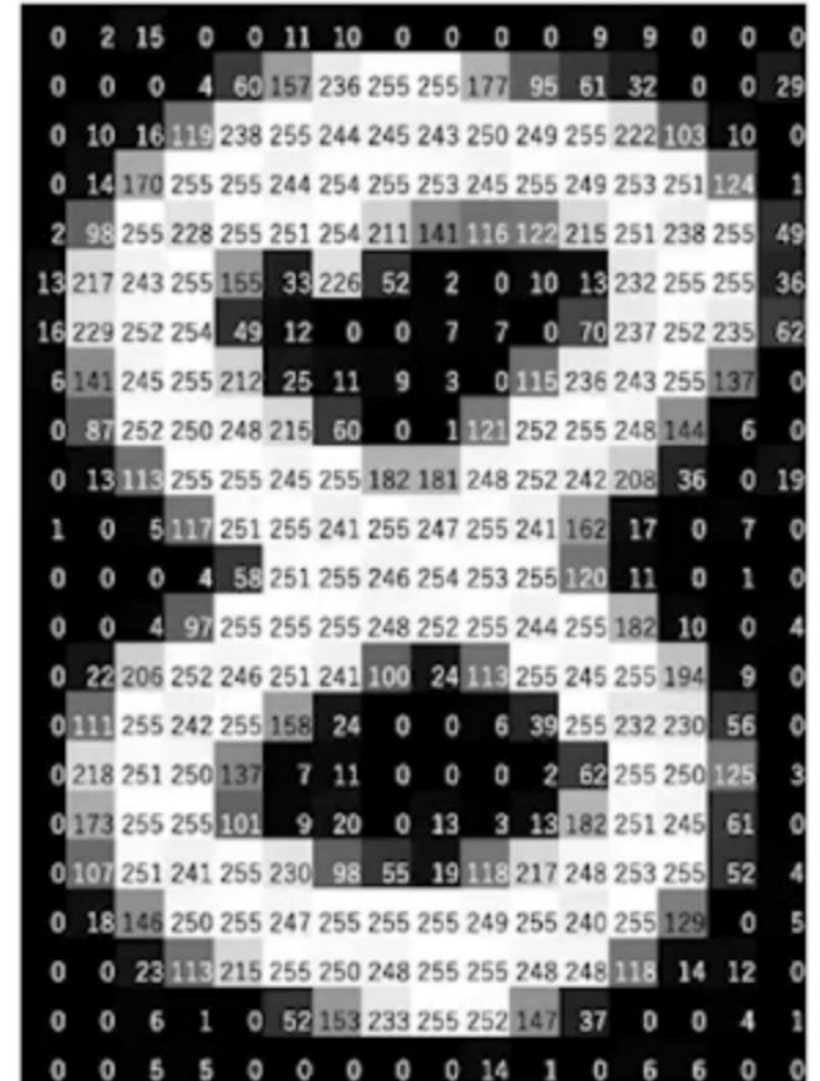
All of these channels or matrices superimpose over one another to form the shape of the image when loaded into a computer. The computer reads this image as:

$$N \times M \times 3 \longrightarrow 6 \times 5 \times 3$$

Grayscale Image



- Grayscale image is one of the digital image categories where every pixel may only be of varying shades of gray without any color information.



Exercise

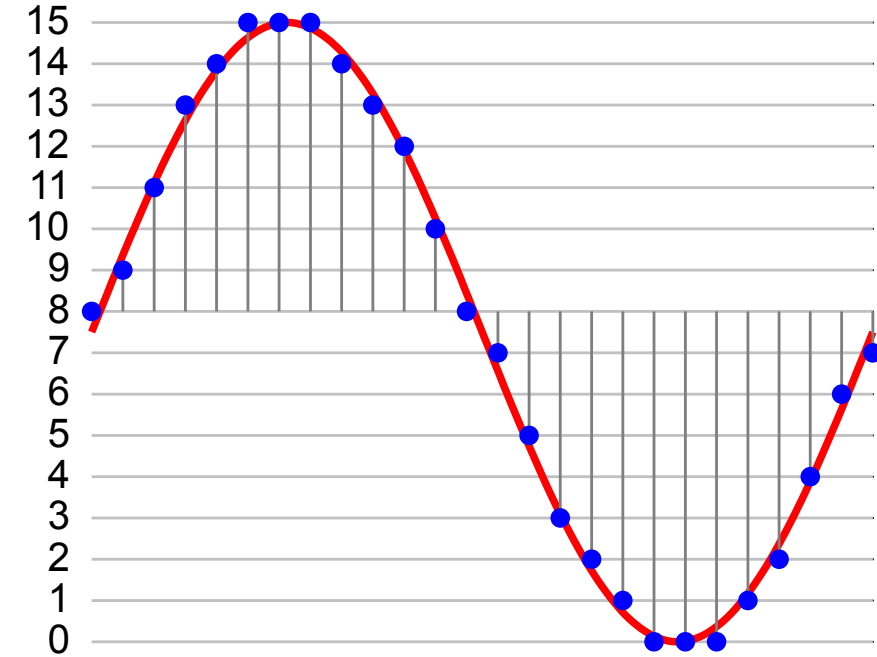


- How many different colors can TrueColor RGB represent?
- Calculate the size of a 1600x4780 pixel large uncompressed picture in 16bit-RGB!
- What happens to the size of the uncompressed picture if an alpha channel is added?
- What happens to the size of the uncompressed picture if it switched to YCbCr color coding?

Audio Data



- Audio is an analog signal
 - Human ear: 20Hz – 20kHz
 - Needs digitization in time and value
- Time domain: Sampling Rate
 - Value taken at certain discrete time intervals
 - Allows for recreation of signal if done fast enough
- Value domain: Bit Quantization
 - Analog value must be represented with n bits
 - More bits allow for better reconstruction
- Recreation of signal by interpolation
 - Use stored bits to modify the intensity of a generic sin wave



Audio Data



- Combination of multiple channels possible
 - Separation by direction/device/type
 - Direction: 1/2/6 channels are mono/stereo/5.1 surround
 - Type: instrument/vocal/background/...
 - Device: violine/piano/harp/trumpet/...

- File size of stored audio is considerable!
 - Compression: MPEG, Dolby, DTS, FLAC, AAC, ...
 - Predictive coding
 - Psychoacoustic coding
 - Remove quiet frequencies
 - Remove certain hard-to-hear frequencies
 - Remove hard-to-resolve directional components
 - Lossy vs. lossless
 - Constant vs. variable

$$\text{bitrate} = \text{samplerate} \times \frac{\text{bits}}{\text{sample}} \times \text{channels}$$

-

Raw Data



- Binary data
 - Handling depends on data type
 - Many of the previous data types is binary data
 - Identification based on file type (extension) and header embedded in beginning
 - Metadata and checksums to confirm content detection
- No clear simple data type
 - Often just referred to by its storage address (pointers)

Task 1



- With which operation can you flip the 5 left-most bits of an 16bit unsigned integer?